



TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH KHOA CÔNG NGHỆ THÔNG TIN

HỆ ĐIỀU HÀNH



ThS. Dương Minh Tuấn

0973887793

Email: dmtuan@ntt.edu.vn

Chương 4

Deadlocks

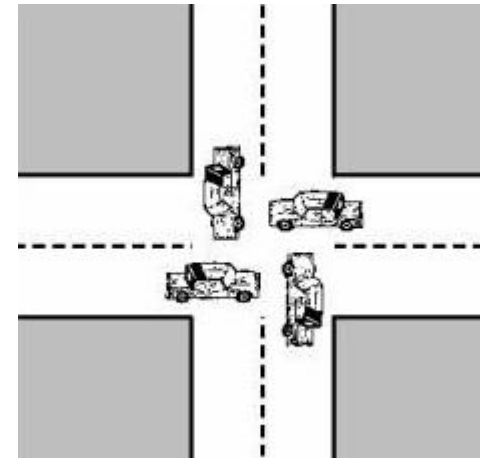
- 1. Giới thiệu về deadlock*
- 2. Các phương thức quản lý deadlock*
- 3. Ngăn chặn deadlock*
- 4. Tránh deadlock*
- 5. Phát hiện deadlock*
- 6. Phục hồi deadlock*



Tổng quan về Deadlock

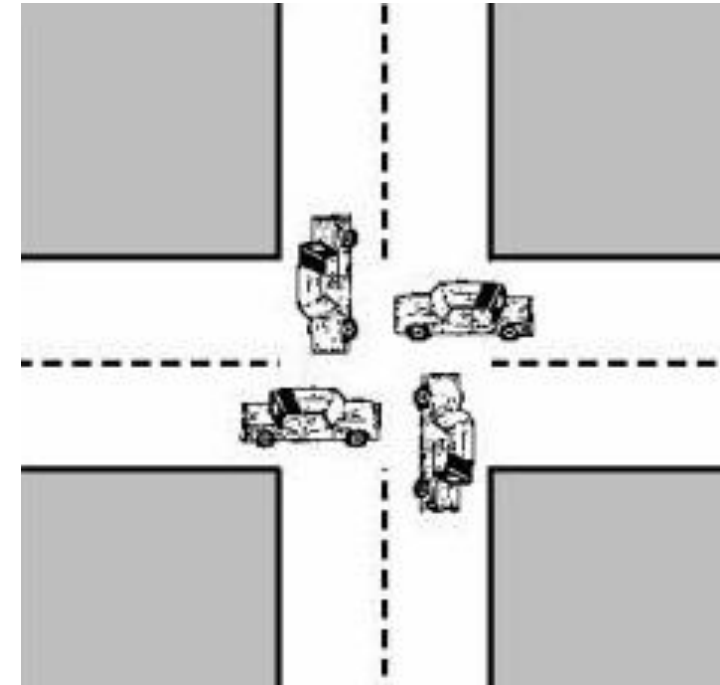
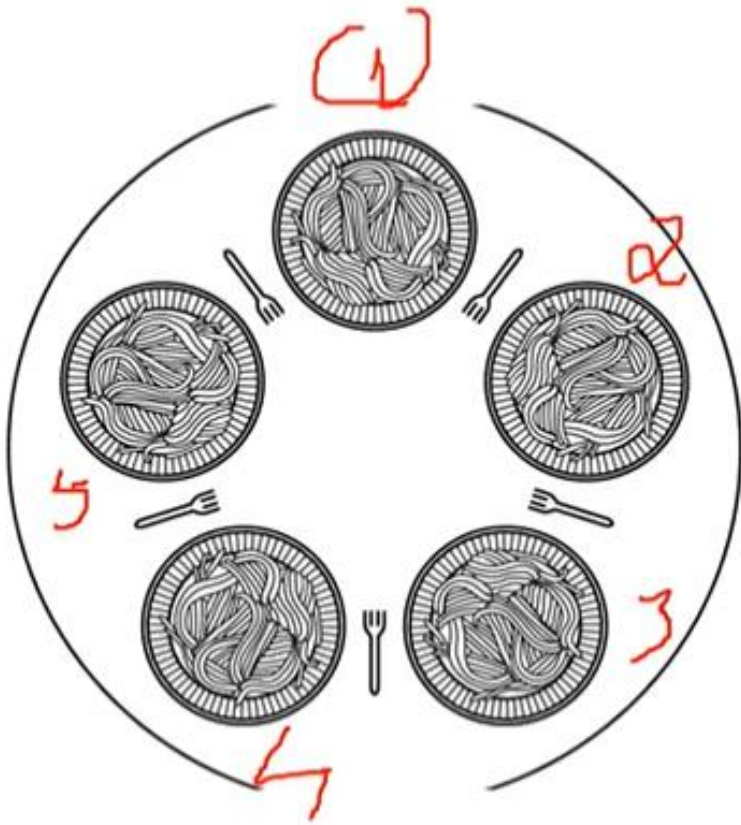
■ Khái niệm Deadlock:

- ▶ Hệ điều hành đa chương: có nhiều tiến trình cạnh tranh chiếm dụng tài nguyên hữu hạn.
- ▶ Một tiến trình **P** yêu cầu tài nguyên **R**, nếu tài nguyên **R** không sẵn dùng tại thời điểm đó, **P** đi vào trạng thái chờ (*waiting*).
- ▶ **Deadlock** (khóa chết / tắc nghẽn) là sự cố mà tiến trình không thoát ra được trạng thái chờ (*waiting*).
 - Tiến trình bị **deadlock** nếu nó đang đợi một sự kiện sẽ không bao giờ xảy ra
 - Tiến trình gọi là **trì hoãn vô hạn định** nếu nó bị trì hoãn một khoảng thời gian dài lặp đi lặp lại.



Tổng quan về Deadlock

- Ví dụ Deadlock:



Tổng quan về Deadlock

■ Lợi ích của việc ngăn chặn Deadlock:

Ngăn chặn Deadlock trong hệ thống có nhiều lợi ích quan trọng, bao gồm:

- ▶ **Đảm bảo tính sẵn sàng của hệ thống:** Deadlock có thể làm cho một hoặc nhiều quy trình bị treo và không thể hoàn thành công việc của mình. Bằng cách ngăn chặn Deadlock, ta đảm bảo rằng hệ thống luôn hoạt động một cách trơn tru và đáng tin cậy.
- ▶ **Tăng hiệu suất và hiệu quả:** Deadlock tiêu tốn tài nguyên và làm giảm hiệu suất của hệ thống. Bằng cách ngăn chặn Deadlock, ta tối ưu hóa việc sử dụng tài nguyên và tăng hiệu suất hoạt động của hệ thống.

Tổng quan về Deadlock

- **Lợi ích của việc ngăn chặn Deadlock:**

- ▶ **Đảm bảo tính công bằng trong việc phân phối tài nguyên:**
Deadlock có thể xảy ra khi có một sự cạnh tranh không công bằng trong việc sử dụng tài nguyên. Bằng cách ngăn chặn Deadlock, ta đảm bảo rằng mọi quy trình và nguồn tài nguyên được phân phối một cách công bằng và hiệu quả.
- ▶ **Tránh mất mát dữ liệu và tình huống không mong muốn:**
Deadlock có thể dẫn đến mất mát dữ liệu hoặc tình huống không mong muốn, gây ảnh hưởng xấu đến hệ thống và dữ liệu. Bằng cách ngăn chặn Deadlock, ta giảm thiểu rủi ro và đảm bảo tính an toàn của dữ liệu.

Tổng quan về Deadlock

■ Lợi ích của việc ngăn chặn Deadlock:

- ▶ **Tăng độ tin cậy của hệ thống:** Deadlock có thể làm hệ thống không ổn định và dễ gặp sự cố. Bằng cách ngăn chặn Deadlock, ta tăng độ tin cậy và sự ổn định của hệ thống.
- ▶ **Tiết kiệm thời gian và nguồn lực:** Việc giải quyết Deadlock yêu cầu thời gian và nguồn lực đáng kể. Bằng cách ngăn chặn Deadlock, ta tránh phải tiêu tốn thêm thời gian và nguồn lực để xử lý và khắc phục Deadlock.
- ▶ **Tóm lại:** việc ngăn chặn Deadlock mang lại nhiều lợi ích quan trọng cho hệ thống, bao gồm tính sẵn sàng, hiệu suất tối ưu, tính công bằng, độ tin cậy và tiết kiệm thời gian và nguồn lực.

Tổng quan về Deadlock

■ Cấp phát tài nguyên cho process:

▶ Tài nguyên (*resource*) trong hệ thống:

- Tài nguyên như: CPU cycle, không gian bộ nhớ, thiết bị I/O, file...
- Bên trong một tài nguyên có thể có một hay nhiều thực thể (*instance*)



▶ *Process* sử dụng tài nguyên theo thứ tự

- Yêu cầu (*request*): process gửi yêu cầu sử dụng tài nguyên và phải chờ nếu yêu cầu không được đáp ứng ngay.
- Sử dụng (*use*): quá trình *process* sử dụng tài nguyên.
- Hoàn trả (*release*): *process* hoàn trả tài nguyên về cho hệ thống

▶ Vai trò của *System calls*:

- Tác vụ *yêu cầu* và tác vụ *hoàn trả* được thực hiện bởi *System Calls*
- *Request / release device*: đáp ứng *yêu cầu* và *thu hồi* truy cập thiết bị.
- *Open / close file*: cho phép *mở* và *đóng* tập tin.
- *Allocate / free memory*: *cấp phát* và *giải phóng* bộ nhớ.

Tổng quan về Deadlock

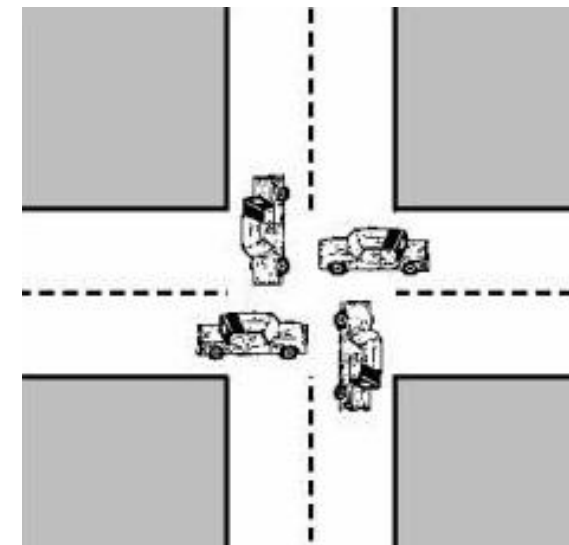
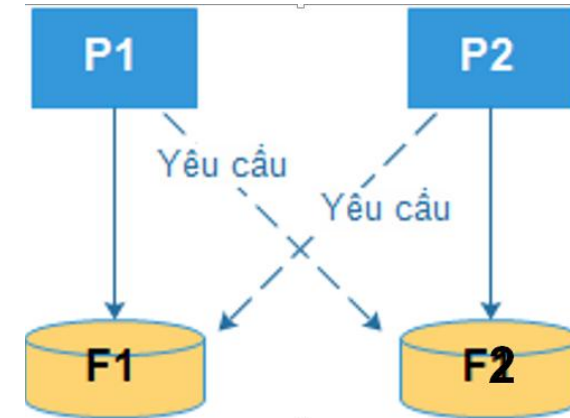
■ Bài toán xảy ra Deadlock:

► Bài toán 1:

- Tiến trình **P1** đang giữ file **F1**.
- Tiến trình **P2** đang giữ file **F2**.
- Khi **P1** yêu cầu file **F2** => **P1** chờ cho đến khi **P2** hoàn trả **F2**.
- Nếu **P2** lại yêu cầu file **F1** => **P2** chờ **P1** hoàn trả **F1** => *Deadlock* xảy ra khi cả 2 chờ lẫn nhau

► Bài toán 2: tiến trình chờ đợi lẫn nhau

- Xe 1 chờ Xe 2 thoát ra để chạy.
- Xe 2 chờ Xe 3
- Xe 3 chờ Xe 4
- Xe 4 chờ Xe 1...

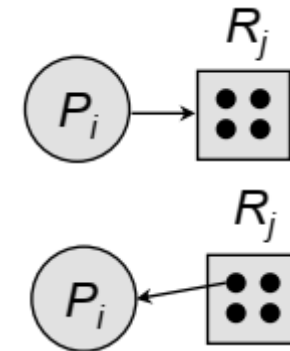


Đồ thị cấp phát tài nguyên - RAG

Mỗi quan hệ giữa các tiến trình và các tài nguyên được mô hình hóa thành **đồ thị** để nhận dạng *Deadlock* dễ dàng hơn.

■ Đồ thị cấp phát tài nguyên:

- ▶ *Resource allocation graph* (**RAG**): là đồ thị có hướng, với tập các đỉnh **V** và tập các cạnh **E** (cạnh: đường nối giữa các đỉnh)
- ▶ Tập đỉnh **V**: có 2 loại:
 - Tập hợp của các tiến trình **P_i** trong hệ thống.
 - Tập hợp của các tài nguyên **R_j** trong hệ thống
- ▶ Tập cạnh **E**: có 2 loại:
 - *Request edge*: hướng từ tiến trình **P_i** yêu cầu tài nguyên **R_j**.
 - *Assignment edge*: hướng từ tài nguyên **R_j** cấp phát cho tiến trình **P_i**.



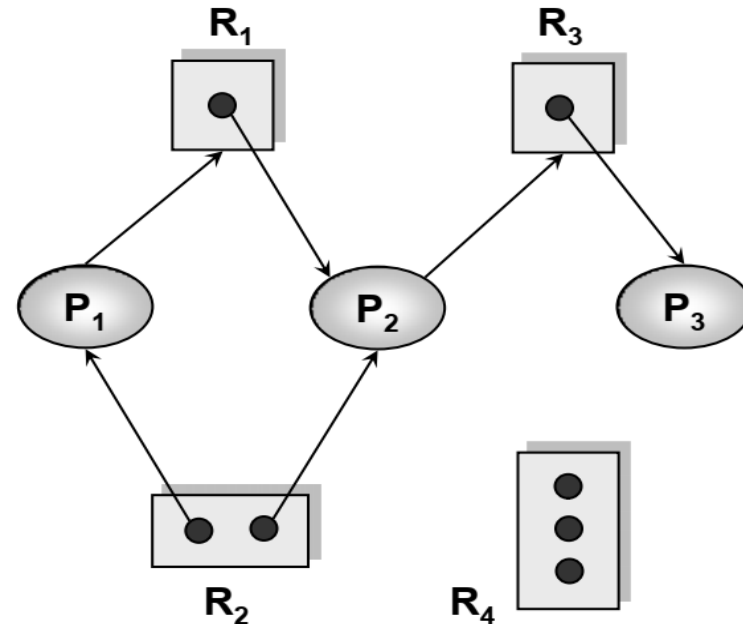
Đồ thị cấp phát tài nguyên - RAG

■ Đánh giá Deadlock từ RAG:

- ▶ Đồ thị RAG không chứa chu trình => không xảy ra *Deadlock*.
- ▶ Đồ thị RAG chứa một hay nhiều chu trình
 - Nếu mỗi loại tài nguyên chỉ có một thực thể => xảy ra *Deadlock*
 - Nếu mỗi loại tài nguyên có nhiều thực thể => có thể xảy ra *Deadlock*

■ Minh họa RAG:

- ▶ Đồ thị RAG bên không xảy ra *Deadlock* vì không chứa chu trình
- ▶ (**R2** có 2 *instance*, cấp phát riêng cho **P1** và **P2**)



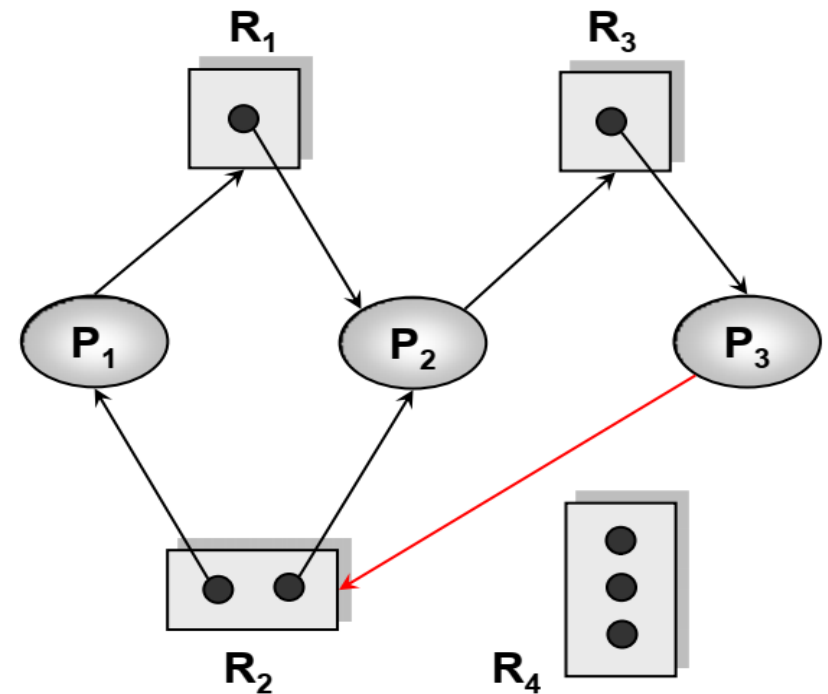
Đồ thị cấp phát tài nguyên - RAG

■ Minh họa RAG:

▶ Đồ thị RAG dưới xảy ra *Deadlock* vì chứa chu trình:

• Chu trình 1: P1-R1-P2-R3-P3-R2-P1

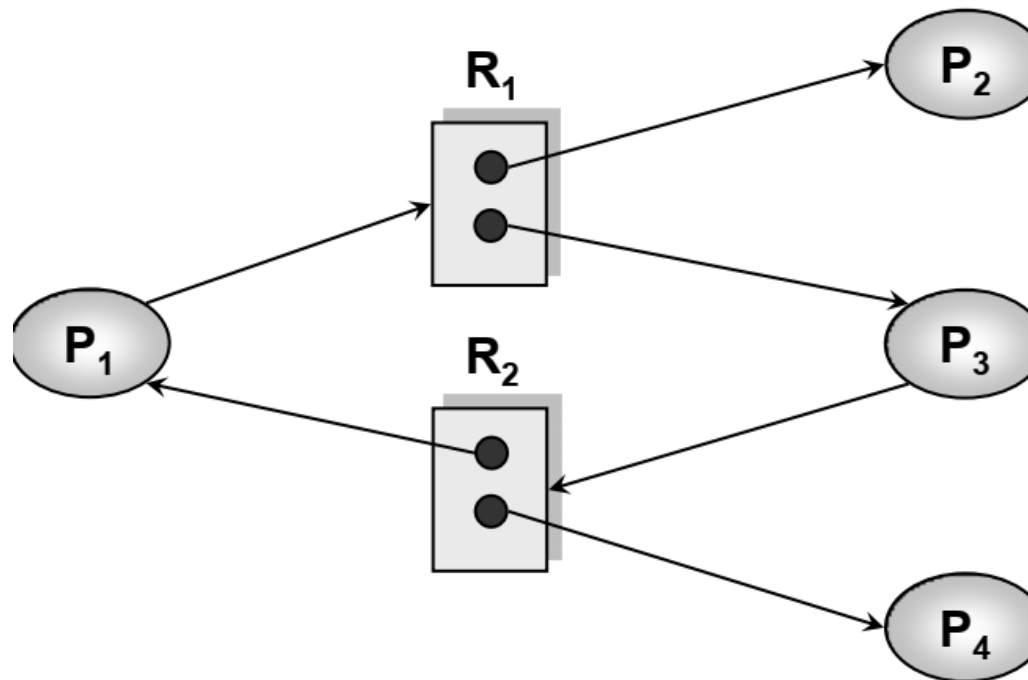
• Chu trình 2: P2-R3-P3-R2-P2



Đồ thị cấp phát tài nguyên - RAG

■ Minh họa RAG:

- ▶ Đồ thị RAG dưới có chứa chu trình (P1-R1-P3-R2-P1).
- ▶ Nhưng *Deadlock* có thể không xảy ra nếu **P4** hoàn trả *instance* cho **R2**



Phương pháp giải quyết deadlock

■ 4 điều kiện (nguyên nhân) đồng thời để Deadlock có thể xảy ra:

▶ **Loại trừ lẫn nhau** (*Mutual Exclusion*):

- Một tài nguyên không thể sử dụng bởi nhiều hơn một tiến trình tại một thời điểm
- Tiến trình **P** được cấp phát tài nguyên **R**.
- Tiến trình khác không sử dụng **R** cho đến khi nó được **P** giải phóng.

▶ **Giữ và chờ** (*Hold and Wait*):

- Các tiến trình giữ tài nguyên và chờ tài nguyên mới
- Tiến trình **P1** đang giữ tài nguyên **R1**.
- **P1** được phép yêu cầu và chờ tài nguyên **R2** do **P2** khác đang giữ.

Phương pháp giải quyết deadlock

■ 4 điều kiện (nguyên nhân) đồng thời để Deadlock có thể xảy ra:

▶ **Cơ chế độc quyền**(*Non-preemption*):

- Các tài nguyên không thể bị đòi lại, chúng chỉ có thể được giải phóng bởi chính tiến trình chiếm giữ chúng
- Khi tiến trình **P1** đang giữ tài nguyên **R1**, nó không cho phép tiến trình khác truy cập **R1** cho đến khi **P1** giải phóng tài nguyên **R1**.

▶ **Tồn tại chu trình chờ**(*Circular wait*):

- Các tiến trình giữ tài nguyên và chờ các tài nguyên bị giữ bởi tiến trình khác, tạo thành một chu trình
- Tiến trình **P1** chờ **P2**, tiến trình **P2** chờ **P3**... tiến trình **Pn** chờ **P1**.
- Chúng nối đuôi chờ nhau, tạo thành vòng tròn chờ => Chờ vô tận.

Phương pháp giải quyết deadlock

■ Các phương pháp giải quyết Deadlock:

▶ Phương pháp ngăn chặn (*preventing*) và tránh né (*avoiding*):

- Ngăn chặn (*Deadlock prevention*): không cho phép xảy ra ít nhất 1 trong 4 điều kiện của *deadlock*.
- Phòng tránh (*Deadlock avoidance*): các *process* cần cung cấp thông tin tài nguyên cần dùng để Hệ điều hành cấp phát hợp lý.

▶ Cho phép – Phát hiện – Phục hồi:

- Hệ điều hành cho phép tiến trình vào trạng thái *deadlock*, nhưng sau đó phát hiện *deadlock* và phục hồi tiến trình đó.

▶ Bỏ qua Deadlock:

- Hệ điều hành xem như *deadlock* không xảy ra trong hệ thống.
- *Deadlock* không được phát hiện => số lượng process bị deadlock tăng dần => hệ thống ngưng hoạt động.

Phương pháp ngăn chặn Deadlock

- **Nguyên tắc:**

- ▶ Ngăn chặn deadlock bằng cách ngăn chặn một trong 4 điều kiện cần của deadlock

- **Ngăn chặn điều kiện *loại trừ tương hỗ (Mutual exclusion)***

- ▶ Đối với tài nguyên không thể chia sẻ (*nonsharable resource*), ví dụ như *printer*:
 - ***Không ngăn chặn được*** vì khi tiến trình **P** chiếm tài nguyên loại này, tiến trình khác phải đợi **P** hoàn trả thì mới truy cập được.
- ▶ Đối với tài nguyên có thể chia sẻ (*sharable resource*), ví dụ như *read-only file*:
 - Việc ngăn chặn là không cần thiết vì tiến trình sẽ hoàn trả tài nguyên ngay sau khi tác vụ ***đọc*** hoàn thành.

Phương pháp ngăn chặn Deadlock

■ Ngăn chặn điều kiện *Giữ và chờ (Hold and wait)*:

▶ Cách 1:

- Mỗi tiến trình yêu cầu toàn bộ tài nguyên cần thiết một lần.
- Nếu có đủ tài nguyên thì hệ thống sẽ cấp phát.
- Nếu không đủ tài nguyên thì tiến trình phải tạm dừng (*blocked*) cho đến khi Hệ điều hành thu hồi đủ tài nguyên.

▶ Cách 2:

- Khi yêu cầu tài nguyên, tiến trình không được giữ tài nguyên nào.
- Nếu tiến trình đang giữ tài nguyên thì phải hoàn trả trước khi yêu cầu tài nguyên khác.

▶ Khuyết điểm:

- Hiệu suất sử dụng tài nguyên thấp.
- Có thể có hiện tượng tiến trình bị đói (*starvation*) tài nguyên.

Phương pháp ngăn chặn Deadlock

■ Ngăn chặn điều kiện *Non-preemption*:

- ▶ Giả định: tiến trình **A** là *non-preemption*, đang giữ tài nguyên **R1** và **A** yêu cầu thêm tài nguyên **R2**
- ▶ **Cách 1:**
 - Hệ thống thu hồi mọi tài nguyên mà **A** đang giữ (tài nguyên **R1**)
 - Hệ thống tiến hành cấp lại tài nguyên **R1** và **R2** cho **A** nếu cả 2 tài nguyên này không có tranh chấp.
- ▶ **Cách 2:** Hệ thống xem tài nguyên mà **A** yêu cầu (tài nguyên **R2**)
 - Nếu **R2** được giữ bởi một tiến trình **B**, đồng thời, **B đang đợi** thêm tài nguyên khác => **R2** được thu hồi để cấp phát cho **A**.
 - Nếu **R2** được giữ bởi một tiến trình **B**, đồng thời, **B không đợi** tài nguyên khác => **A** phải đợi và tài nguyên **R1** bị thu hồi.

Phương pháp ngăn chặn Deadlock

■ Ngăn chặn điều kiện *Circular wait*:

▶ Nguyên lý:

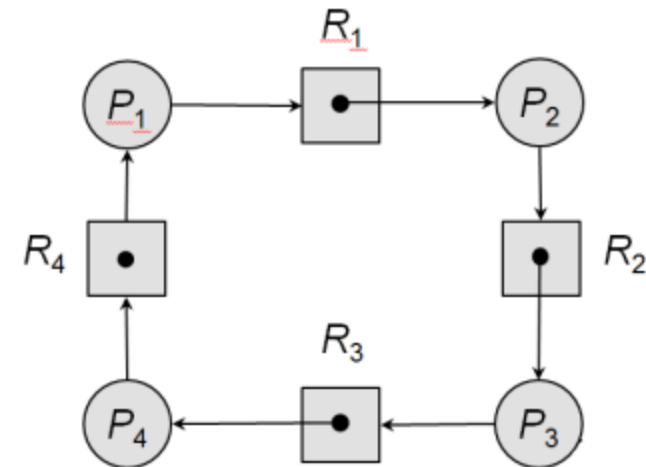
- Tất cả tài nguyên trong hệ thống được gán một thứ tự:
Tập hợp tài nguyên: $R = \{R_1, R_2, \dots, R_n\}$
- Mỗi tiến trình chỉ có thể yêu cầu thực thể (*Instance*) của một loại tài nguyên theo thứ tự tăng dần

▶ Giải pháp:

- Tiến trình **P** đang giữ các tài nguyên, nếu **P** yêu cầu thêm tài nguyên R_j , thì **P** phải trả lại các tài nguyên R_i với $F(R_i) > F(R_j)$

▶ Minh họa: hình bên là một *Circular wait*.

- **P1** đang giữ R_4 ($i=4$).
- Nếu **P1** yêu cầu R_1 ($j=1$), **P1** phải hoàn trả R_4 ($i=4 > j$)
- => mất *Circular wait*



Phương pháp tránh Deadlock

■ Đặc điểm của phương pháp tránh (*Deadlock avoidance*)

- ▶ Hệ điều hành đặt ra các quy tắc sắp xếp thứ tự tiến trình thực thi nhằm tránh né hiện tượng tắc nghẽn (*Deadlock avoidance*):
 - *Deadlock Prevention* sử dụng tài nguyên không hiệu quả.
 - *Deadlock avoidance* vẫn đảm bảo hiệu suất sử dụng tài nguyên tối đa đến mức có thể
- ▶ Phương pháp *Deadlock avoidance*:
 - Mỗi tiến trình phải khai báo số lượng tài nguyên tối đa cần để thực hiện công việc.
 - Hệ điều hành sẽ kiểm tra trạng thái cấp phát tài nguyên để đảm bảo hệ thống không rơi vào deadlock.
- ▶ Trạng thái cấp phát tài nguyên được định nghĩa dựa trên:
 - Số tài nguyên còn lại,
 - Số tài nguyên đã được cấp phát.
 - Yêu cầu tối đa của các tiến trình.

Phương pháp tránh Deadlock

■ Trạng thái safe (an toàn) và unsafe (không an toàn)

- ▶ Hệ điều hành sắp xếp các tiến trình (**P1, P2,...,Pn**) theo chuỗi.
- ▶ Trạng thái của hệ thống được gọi là an toàn (*safe*) nếu tồn tại một chuỗi thứ tự tiến trình an toàn
- ▶ Chuỗi tiến trình (**P1, P2,...,Pn**) là **an toàn** nếu yêu cầu **tối đa** về tài nguyên mọi tiến trình **Pi** (**i = 1...n**) được thỏa bởi:
 - Tài nguyên mà hệ thống đang có sẵn sàng (*Available resource*)
 - Cộng với tài nguyên mà tất cả các tiến trình **Pj** (**j < i**) đang giữ.

Số yêu cầu tài
nguyên tối đa của **Pi**



Số tài nguyên
đang có sẵn



Số tài nguyên thu
hồi từ các **Pj** (**j < i**)

- ▶ Một trạng thái của hệ thống được gọi là **không an toàn** (*unsafe*) nếu không tồn tại một chuỗi an toàn.

Phương pháp tránh Deadlock

■ Minh họa chuỗi tiến trình *Safe* và *Unsafe*

- ▶ Giả sử, hệ thống có **12** files (tài nguyên) và 3 tiến trình **A, B, C**
- ▶ Tại thời điểm **t₀** trạng thái sử dụng *file* các tiến trình như sau:

Tiến trình	Cần tối đa	Đang giữ	Cần thêm
A	10	5	5
B	4	2	2
C	9	2	7

- Số *file* (R) còn dư (*available*) là: **12 – (5+2+2) = 3** (*files*)
- ▶ Nếu *Hệ điều hành* lập chuỗi tiến trình theo thứ tự: **A-B-C** thì:
 - Xét tiến trình **A** (**i=1**) (*i* là thứ tự của tiến trình trong chuỗi)
 - Tiến trình có thể thu hồi *file* cho **A** : **không có** (do **A** có thứ tự **i = 1**)
 - Tiến trình **A** cần thêm **5** (*files*) **nhiều hơn 3** (*files* dư)
 - => chuỗi tiến trình theo thứ tự: **A-B-C** là **không an toàn**.

Phương pháp tránh Deadlock

■ Minh họa chuỗi tiến trình *Safe* và *Unsafe*

- ▶ Tại thời điểm t_0 trạng thái sử dụng *file* các tiến trình như sau:

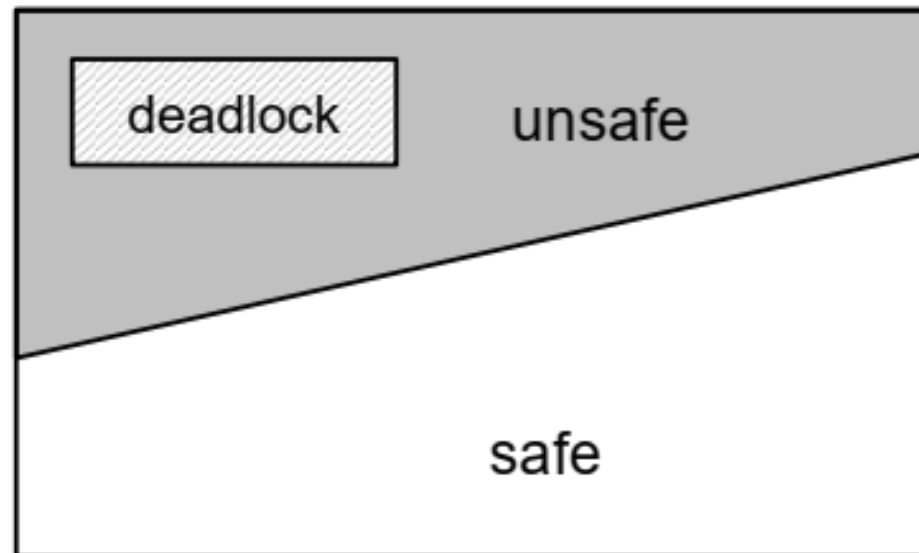
Tiến trình	Cần tối đa	Đang giữ	Cần thêm
A	10	5	5
B	4	2	2
C	9	2	7

- ▶ Nếu *Hệ điều hành* lập chuỗi tiến trình theo thứ tự: **B-A-C** thì:
 - Xét tiến trình **C** (**i=3**):
 - Tiến trình có thể thu hồi *file* cho **C** là **A** và **B**: **5+2= 7** (*files*)
 - Tiến trình **C** cần thêm **7** (*files*) **ít hơn** (**3** (*files dư*) + **7** (*files của A và B*))
 - => chuỗi tiến trình theo thứ tự: **B-A-C** là **an toàn**.

Phương pháp tránh Deadlock

■ Ý tưởng tránh Deadlock bằng Safe và Unsafe:

- ▶ Sắp xếp trật tự của các tiến trình trong hàng đợi sao cho:
 - Hệ thống đang ở trạng thái **safe** -> **không deadlock**
 - Hệ thống đang ở trạng thái **unsafe** -> có thể dẫn đến **deadlock**.
- ▶ Tránh **deadlock** bằng cách bảo đảm hệ thống không đi đến trạng thái **unsafe**.



Phát hiện Deadlock

Đối với một số Hệ điều hành cho phép deadlock xảy ra, việc phát hiện Deadlock sẽ giúp hệ thống phục hồi cho tiến trình bị sự cố đó.

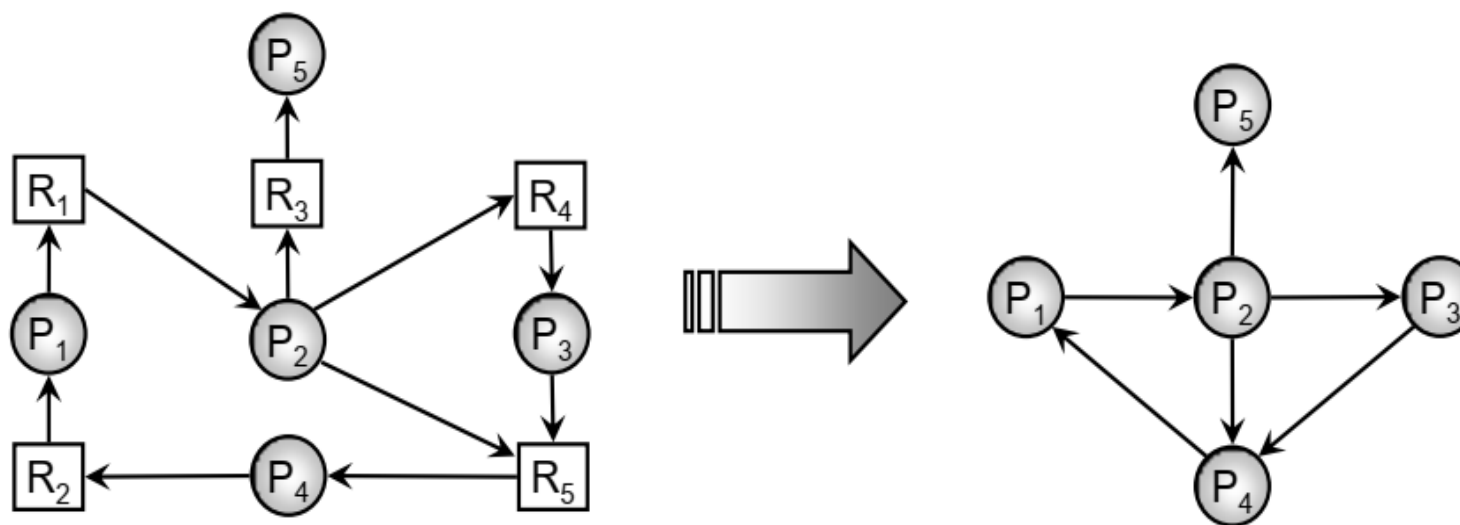
■ Giải thuật phát hiện Deadlock:

- ▶ Thường sử dụng Đồ thị sử dụng tài nguyên (*Resource allocation graph* - **RAG**)
- ▶ Giải thuật phát hiện Deadlock thiết kế cho 2 trường hợp:
 - Trường hợp mỗi loại tài nguyên có 1 thực thể / thể hiện (*instance*)
 - Trường hợp mỗi loại tài nguyên có nhiều thực thể (*instance*)
- ▶ Giải thuật Đồ thị chờ (*Wait for Graph*): dùng cho trường hợp tài nguyên có 1 thực thể.
- ▶ Giải thuật Nhà băng (*Banker*): dùng cho trường hợp tài nguyên có nhiều thực thể.

Phát hiện Deadlock

■ Giải thuật đồ thị chờ tài nguyên (*Wait for Graph*)

- ▶ *Wait for Graph* là đồ thị dẫn xuất từ RAG bằng cách:
 - Loại bỏ các nút biểu thị tài nguyên (R)
 - Các cạnh nối các tiến trình, có hướng tương ứng hướng yêu cầu tài nguyên



- ▶ Nếu đồ thị chờ tài nguyên (*Wait for Graph*) có chu trình => **có xảy ra *Deadlock***.

Phát hiện Deadlock

■ Giải thuật Nhà băng (Banker)

- Có tên như trên vì giải thuật này có thể được sử dụng trong hệ thống nhà băng để đảm bảo rằng nhà băng không bao giờ phân phối quá số tiền khả dụng của nó đến mức mà nó có thể thỏa mãn mọi yêu cầu từ khách hàng
- Khi một tiến trình mới đi vào hệ thống nó phải khai báo số lượng tối đa các thực thể của mỗi loại tài nguyên mà nó có thể cần đến. Số này có thể vượt quá tổng tài nguyên trong hệ thống
- Khi user yêu cầu tài nguyên hệ thống phải xác định liệu sự phân phối có thể giữ hệ thống trong trạng thái an toàn không
- Nếu có: phân phối tài nguyên
- Nếu không: tiến trình phải chờ đến khi các tiến trình khác giải phóng đủ tài nguyên

Phục hồi Deadlock

■ Các phương pháp phục hồi Deadlock

- ▶ Phục hồi Deadlock được thực hiện bởi:
 - Người quản lý Hệ điều hành: dùng các công cụ quản lý tiến trình như *Task Manager, ps, kill...*
 - Do Hệ điều hành tự giải quyết.
- ▶ Hệ điều hành tự phục hồi bằng cách phá deadlock:
 - Giải pháp chấm dứt tiến trình.
 - Giải pháp thu hồi tài nguyên.
 - Giải pháp Rollback (quay lại từ đầu)

■ Giải pháp chấm dứt tiến trình:

- ▶ Chấm dứt tất cả tiến trình liên quan *deadlock*. Hoặc:
- ▶ Chấm dứt lần lượt các tiến trình cho đến khi không còn *deadlock*

Phục hồi Deadlock

■ Giải pháp thu hồi tài nguyên.

- ▶ Tạm dừng tiến trình P_i có liên quan deadlock.
- ▶ Thu hồi tài nguyên của $P_i \Rightarrow$ cấp cho P_j
- ▶ Khi P_j sử dụng tài nguyên xong $\Rightarrow$ thu hồi $\Rightarrow$ cấp lại cho P_i .

■ Giải pháp Rollback:

- ▶ Xác định một tiến trình P đang giữ tài nguyên mà tiến trình Q đang bị *deadlock* cần.
- ▶ *Rollback* tiến trình P về một thời điểm mà nó chưa truy cập tài nguyên này.
- ▶ Cấp phát tài nguyên này cho tiến trình Q .

